Title: Predicting High-Selling Video Games Using Machine Learning

Name: *Rafid Hasan — T00695541*

Course: COMP 4980 — Special Topics: Machine Learning

Date: 27-Nov-2025

Data Description

Dataset: Video Game Sales — Kaggle (vgsales.csv)
Link: https://www.kaggle.com/datasets/gregorut/videogamesales

The dataset that I used in this project is the *Video Game Sales* dataset from Kaggle, which is pretty popular in Kaggle, having 2.62M views & 744K downloads. It contains historical information about 16,598 video games, covering different consoles, genres, publishers, and sales across major regions. Each row represents one game and includes the following key attributes:

- Name – title of the video game
- Platform – the console it was released on (e.g., PS2, Xbox360, Wii, DS)
- Genre – category such as Action, Sports, RPG, Shooter, etc.
- Publisher – the company that released the game
- Year – release year
- Sales Columns – sales (in millions) for:
    - North America (NA_Sales)
    - Europe (EU_Sales)
    - Japan (JP_Sales)
    - Other regions
    - Global_Sales (worldwide total)

Because Global_Sales is a continuous variable, I transformed it into a binary classification target to fit the course requirements:

- High_Sales = 1 → very high-selling games
- High_Sales = 0 → all other games

After cleaning the data, handling missing release years, and one-hot encoding the categorical features (platforms, genres, publishers), the dataset expanded to over 70 features.

Why This Dataset Is Suitable

This dataset is:

- Clean and structured: No advanced parsing needed.
- Mixed in data types: Categorical + numerical, which is ideal for ML.
- Large enough for meaningful models: 16k rows is perfect for training but not too big to cause performance issues.

The dataset is purely historical, with data ranging roughly from the 1980s to 2016, so it does not include modern titles. Still, it provides a long-term view of the gaming industry and allows patterns to be analyzed across decades.

I selected this dataset because I genuinely love video games, and I was interested in what factors make a game successful in terms of global sales. Even though the dataset is old and does not include newer platforms like Nintendo Switch, PS5, or Xbox Series X, the analysis was still very engaging. Working with this dataset also made me curious about modern gaming trends and how they might differ from the patterns we observed in the early 2000s.

Because gaming is a big part of my life, this project felt more natural and enjoyable. It also motivated me to explore how machine learning could be applied to the gaming industry, such as predicting sales, identifying high-performing genres, or understanding regional trends.

Since this dataset only goes up to around 2016, it represents a past era of gaming. The industry has changed a lot since then — digital downloads, live-service games, subscription models (Game Pass), and the rise of mobile gaming have all reshaped sales patterns.

If I had more recent data, I could compare trends such as:

- the dominance of Nintendo Switch exclusives
- the steep rise of mobile gaming revenue
- the shift from physical sales to digital distribution
- the impact of online multiplayer and free-to-play models

Data Analysis

This section summarizes the initial analysis of the Video Game Sales dataset. We will try to understand the structure of the data, examine key variables, identify patterns, and prepare the dataset for machine learning experiments

Overview of Dataset Shape and Structure

The first step was to load the dataset and check its basic structure. The dataset contains 16,598 rows and several important columns, including Name, Platform, Genre, Publisher, Year, and sales across different regions.

```
df = pd.read_csv("vgsales.csv")

df.head()
```

	Rank	Name	Platform	Year	Genre	Publisher	NA_Sales	EU_Sales	JP_Sales	Other_Sales	Global_Sales
0	1	Wii Sports	Wii	2006.0	Sports	Nintendo	41.49	29.02	3.77	8.46	82.74
1	2	Super Mario Bros.	NES	1985.0	Platform	Nintendo	29.08	3.58	6.81	0.77	40.24
2	3	Mario Kart Wii	Wii	2008.0	Racing	Nintendo	15.85	12.88	3.79	3.31	35.82
3	4	Wii Sports Resort	Wii	2009.0	Sports	Nintendo	15.75	11.01	3.28	2.96	33.00
4	5	Pokemon Red/Pokemon Blue	GB	1996.0	Role-Playing	Nintendo	11.27	8.89	10.22	1.00	31.37

This shows the first few rows of the dataset, confirming the presence of key attributes such as Name, Platform, Genre, Year, and Global_Sales.

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 16598 entries, 0 to 16597
Data columns (total 11 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   Rank          16598 non-null  int64
 1   Name          16598 non-null  object
 2   Platform      16598 non-null  object
 3   Year          16327 non-null  float64
 4   Genre         16598 non-null  object
 5   Publisher     16540 non-null  object
 6   NA_Sales      16598 non-null  float64
 7   EU_Sales      16598 non-null  float64
 8   JP_Sales      16598 non-null  float64
 9   Other_Sales   16598 non-null  float64
 10  Global_Sales  16598 non-null  float64
dtypes: float64(6), int64(1), object(4)
memory usage: 1.4+ MB
```

This highlights:

- Data types (string, integer, float)
- Missing values
- Number of entries

During inspection, I noticed missing values in the Year column, and some publisher names were inconsistent or rare. These were cleaned later during preprocessing.

Descriptive Statistics

I used df.describe() to get a quick summary of the numerical columns.
Global_Sales ranged from 0.01 to more than 80 million, which confirms that the dataset has extreme outliers. North America and Europe sales were generally higher than Japan.

	count	unique	top	freq	mean	std	min	25%	50%	75%	max
Rank	16598.0	NaN	NaN	NaN	8300.605254	4791.853933	1.0	4151.25	8300.5	12449.75	16600.0
Name	16598	11493	Need for Speed: Most Wanted	12	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Platform	16598	31	DS	2163	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Year	16327.0	NaN	NaN	NaN	2006.406443	5.828981	1980.0	2003.0	2007.0	2010.0	2020.0
Genre	16598	12	Action	3316	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Publisher	16540	578	Electronic Arts	1351	NaN	NaN	NaN	NaN	NaN	NaN	NaN
NA_Sales	16598.0	NaN	NaN	NaN	0.264667	0.816683	0.0	0.0	0.08	0.24	41.49
EU_Sales	16598.0	NaN	NaN	NaN	0.146652	0.505351	0.0	0.0	0.02	0.11	29.02
JP_Sales	16598.0	NaN	NaN	NaN	0.077782	0.309291	0.0	0.0	0.0	0.04	10.22
Other_Sales	16598.0	NaN	NaN	NaN	0.048063	0.188588	0.0	0.0	0.01	0.04	10.57
Global_Sales	16598.0	NaN	NaN	NaN	0.537441	1.555028	0.01	0.06	0.17	0.47	82.74

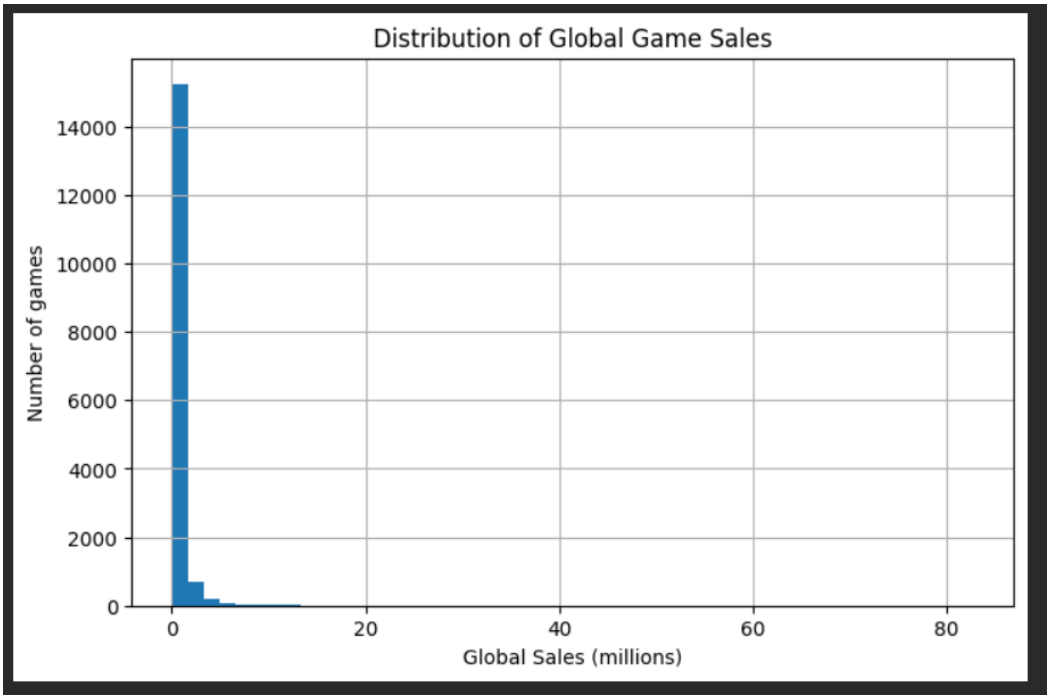
Key observations from descriptive stats:

- Most games sell less than 1 million copies globally.
- Only a small number of games achieve extremely high sales (e.g., Wii Sports, Mario Kart).
- Japan's sales tend to be lower compared to NA and EU, reflecting regional industry differences.
- Year ranges from early 1980s to 2016.

These observations helped justify converting Global_Sales into a binary high/low label, because the raw distribution is heavily skewed.

Distribution of Global Sales

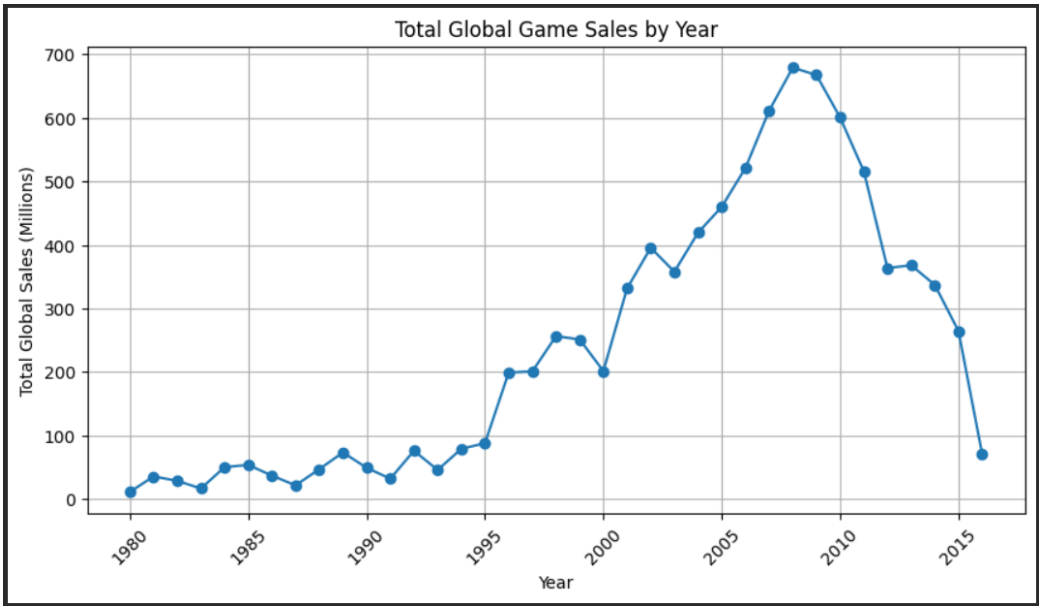
Global sales are not normally distributed. Instead, they have a long right tail, meaning only a few games sell extremely well.



The histogram shows that most games fall into the 0–1 million sales range. This imbalance explains why predicting exact sales is difficult and why I later created a binary "High_Sales" target.

Sales Over Time

To understand how game popularity evolved, I plotted Global_Sales by Year.



What the plot reveals:

- Sales peaked around 2006–2010, matching the popularity of Wii, Xbox 360, and PS3.

- After 2012, overall sales decrease in the dataset because newer platforms (PS5, Switch) are not included.

This supports the idea that the dataset represents an older generation of gaming trends.

Correlation Analysis

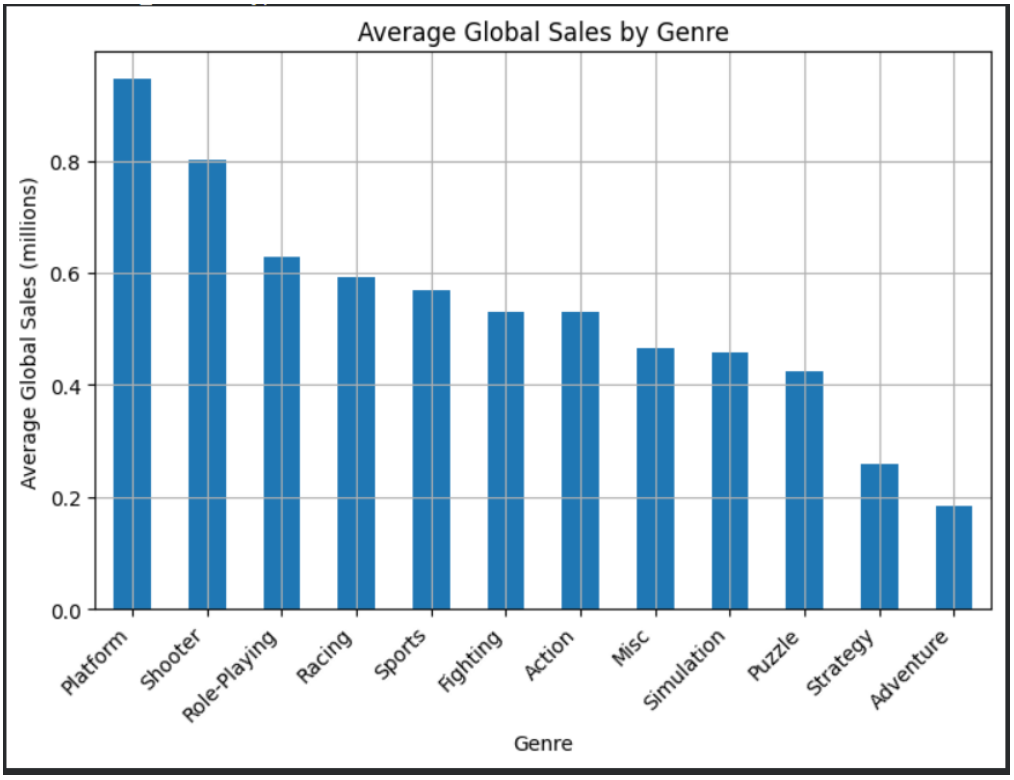
Since most variables are categorical, correlations are limited for numerical columns. The strongest positive correlations were:

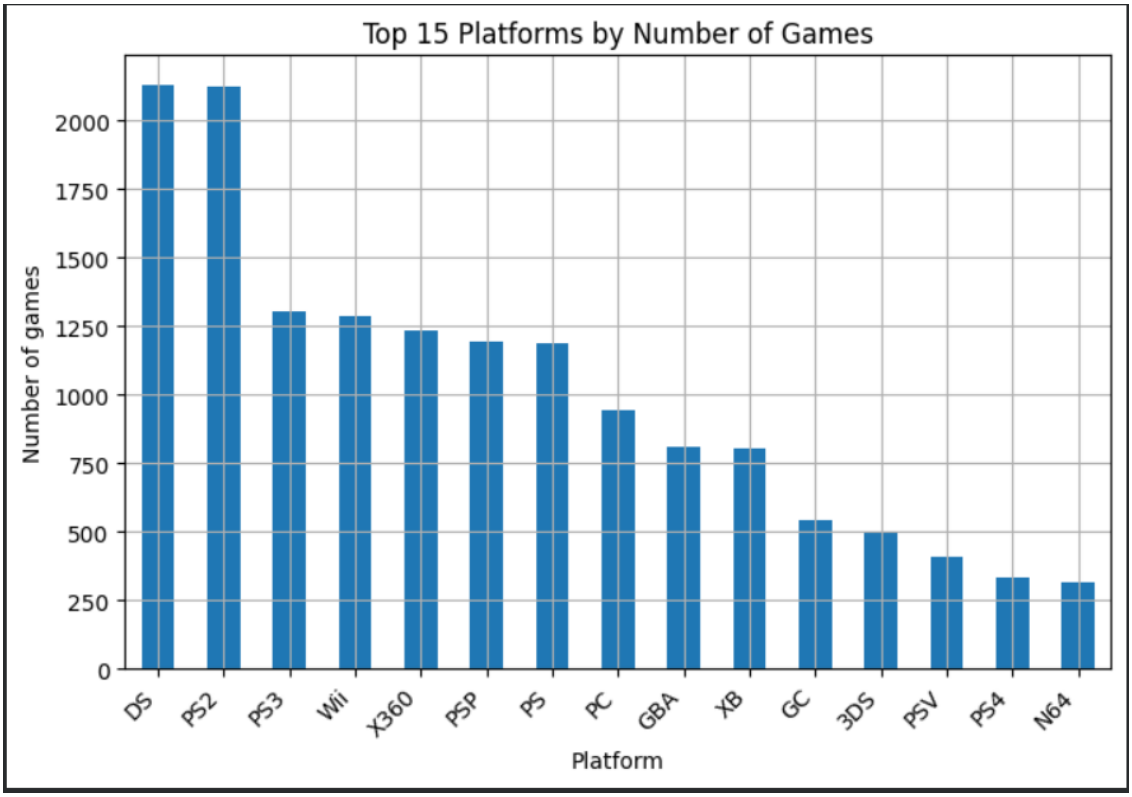
- NA_Sales ↔ EU_Sales
- Global_Sales ↔ NA_Sales
- Global_Sales ↔ EU_Sales

Games that perform well in North America also tend to perform well globally. Japan behaves differently—its correlation with Global_Sales is weaker, which matches the regional preferences in genres and platforms.

Distribution of Genres and Platforms

I analyzed the distribution of game genres and platforms to see which ones dominate the dataset.





Key findings:

- Action and Sports are the most common genres.
- DS, PS2, and Wii appear frequently because they were extremely popular during the dataset era.
- Rare platforms (e.g., 3DO, TG16) appear very few times, which later affected model performance.

Summary of Data Analysis Findings

From the exploration phase, I learned:

- Global Sales are highly imbalanced, with very few high-selling games.
- Regional sales correlate strongly with global performance.
- Year, Platform, Genre, and Publisher seem to play major roles in explaining game success.
- The dataset represents older gaming trends, which limits its representation of modern titles.

- The combination of categorical features and skewed sales values strongly justifies using classification instead of regression.

Data Exploration

The goal of the data exploration phase was to understand the structure of the dataset beyond simple descriptive statistics. In this section, I used the following techniques:

1. **Principal Component Analysis (PCA)**
2. **Decision Tree exploration**

Both methods helped me understand how the encoded features behave, how much information they capture, and which early relationships in the data are worth investigating further.

Principal Component Analysis (PCA)

Since the cleaned dataset has more than 70 features, PCA was used to reduce dimensionality and examine how much variance the main components capture.

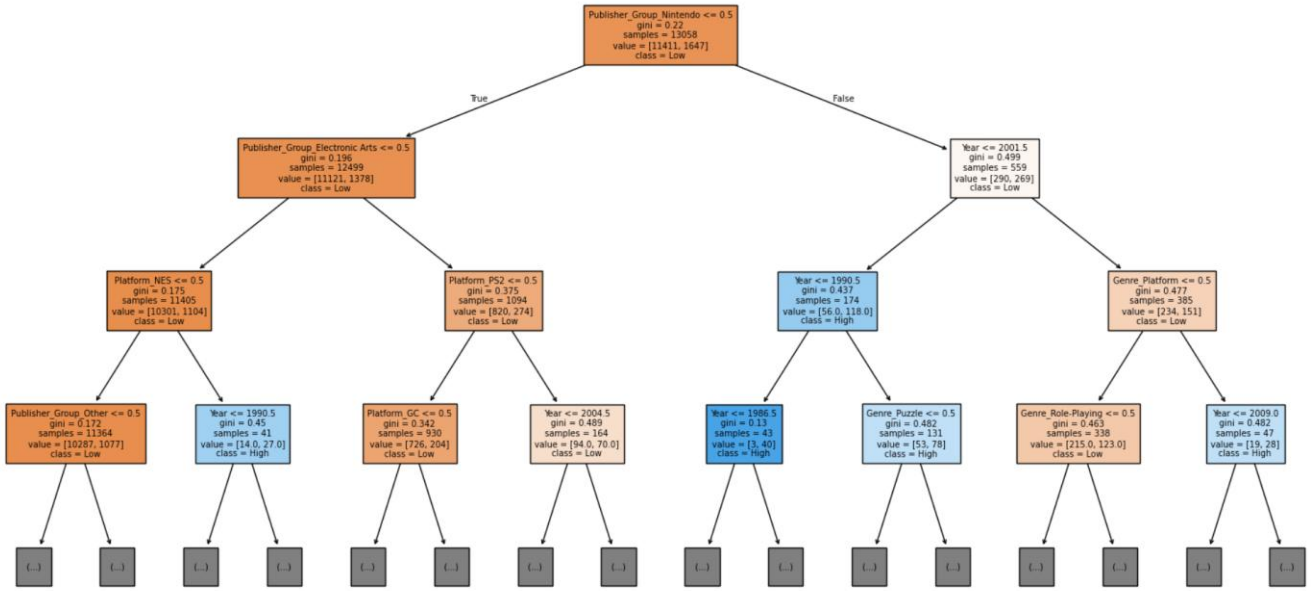
Findings from PCA:

- The cumulative explained variance curve showed that about 10 principal components were enough to explain 96% of the variance.
- This means that although the dataset originally had 70+ features, most of the useful information is concentrated in a much smaller number of underlying patterns.
- This also reflects real-world gaming patterns: certain publishers and platforms tend to dominate sales, creating strong correlations in the encoded data.

PCA helped confirm that the dataset has a lot of overlap in information. Even though there are many platform and publisher categories, only a few combinations truly influence game sales. This validated our choice of using ensemble models (like Random Forest and XGBoost) later on, since they perform well on high-dimensional but correlated datasets.

Decision Tree Exploration

To understand early relationships between features and the target variable, I trained a small decision tree classifier with depth=4. This provided a simple and interpretable visualization.



The decision tree helped me understand the “logic” the model uses when trying to classify a high-selling game. It was interesting to see how much importance publishers and platforms have compared to genres. It also showed that the dataset contains strong structure even before applying more advanced models. This early exploration guided the later experiments and helped me focus on the right features.

Summary of Data Exploration

From PCA and the decision tree analysis, I learned:

- A small number of components explain most of the variance → the dataset is structured and correlated.
- Publishers and platforms appear consistently important.
- Genre and year are relevant but secondary.
- PCA and decision tree results align, indicating stable patterns in the dataset.
- This exploration helped shape the experimental plan, especially the decision to use ensemble models like Random Forest and XGBoost.

Experimental Method

The goal of this phase was to build and refine a machine learning system capable of predicting whether a video game becomes a high-selling title. This section describes my hypothesis, the machine learning models I tested, the preprocessing steps, and the different experiments I performed.

My hypothesis was:

A game's publisher, platform, genre, and release year contain enough information to predict whether it will be a high-selling game.

Ensemble models (Random Forests and Gradient Boosting) should perform better than simpler models because the dataset is high-dimensional and mostly categorical.

Since the dataset was heavily skewed toward low-selling games, I expected accuracy to be misleading and used metrics like F1-score and recall to evaluate the minority class (high-selling games).

Preprocessing

Before training the models, I applied these preprocessing steps:

1. Cleaned missing values, especially in the Year column.
2. Converted Global_Sales into a binary target (High_Sales = 1 or 0).
3. One-hot encoded categorical fields: Platform, Genre, Publisher.
4. Scaled numerical features (for MLP and GradientBoost).
5. Split data into train/test sets using stratified sampling due to class imbalance.

Models Selected for Testing

I experimented with several models:

1. Random Forest Classifier

- Good for high-dimensional categorical datasets.
- Performs implicit feature selection.
- Robust to noise and overfitting.

2. XGBoost

- Known for strong performance on structured tabular data.

- Handles non-linear interactions very well.
- Allows fine-grained control via hyperparameters.

3. Multi-Layer Perceptron (MLP)

- A neural network capable of learning complex patterns.
- Sensitive to feature scaling and class imbalance.
- Useful for comparison with non-neural methods.

Initial Experiments (Baseline Models)

I first trained simple "default" versions of each model to establish a baseline.

Baseline Random Forest

- Accuracy: ~0.86
- Weak recall for high-selling games
- Good starting point, but needed tuning for imbalance

Baseline XGBoost

- Accuracy: ~0.88
- Better recall than Random Forest
- Already showed strong performance

Baseline MLP

- Unstable performance
- Sensitive to imbalance
- Required scaling and tuning

These early results confirmed my hypothesis: ensemble models outperform simpler approaches on this dataset.

Hyperparameter Tuning (GridSearchCV)

To refine the models, I used GridSearchCV with 3-fold cross-validation.
I tuned the following:

For Random Forest:

- n_estimators
- max_depth
- max_features (sqrt, log2)
- min_samples_split
- min_samples_leaf

Best parameters found:

n_estimators = 400, max_features = 'sqrt', min_samples_split = 2, min_samples_leaf = 1, max_depth = None

Random Forest accuracy improved slightly, but recall remained low.

For XGBoost:

- n_estimators (200–400)
- max_depth (6–10)
- learning_rate
- subsample
- colsample_bytree
- gamma

Best parameters:

400 trees, max_depth=10, learning_rate=0.1, subsample=0.8, colsample_bytree=0.8, gamma=0

This significantly improved macro F1-score and recall for high-selling games.

XGBoost became the best model overall.

For MLP (Neural Network):

- Hidden layer sizes
- Activation (relu)
- Learning rate
- Batch size
- Max iterations

Even with tuning, MLP underperformed compared to XGBoost and Random Forest.
It struggled with imbalanced classes and large one-hot encoded data.

Imbalance Handling: SMOTE Experiment

Because only a small fraction of games are high-selling, I tested SMOTE (Synthetic Minority Oversampling Technique) to improve recall.

SMOTE + XGBoost

- Accuracy decreased
- Recall for high-selling games improved significantly
- Macro F1 became more balanced across classes

This experiment helped me understand the trade-off between accuracy and fairness toward the minority class.

Lab Diary Summary (Major Changes & Why)

Experiment / Change	**Why I Tried It**	**What I Learned**
Baseline models	Establish starting point	Ensemble models perform best
One-hot encoding	Required for tree models	Increased dimensionality but worked well
Scaling for MLP	Neural nets require normalization	Improved but still worse than XGBoost

Experiment / Change	Why I Tried It	What I Learned
GridSearch for RF	Improve performance	Only small improvements
GridSearch for XGBoost	Expected best model	Confirmed: best F1 and recall
SMOTE	Fix imbalance	Improved minority recall
Feature importance	Understand patterns	Publisher and platform dominate

Final Method Selection

After all experiments, I selected XGBoost with tuned hyperparameters as the final model because:

- It provided the highest accuracy (≈ 0.88)
- It produced the best macro F1 score
- It identified the minority class much better than Random Forest
- Feature importance interpretations were meaningful

SMOTE-XGBoost was also useful for understanding fairness trade-offs, but not selected as the final deployment model due to lower accuracy.

This phase felt like the “real machine learning work,” because I had to constantly try new ideas, configure parameters, interpret results, and refine the model. Each experiment taught me something — especially about class imbalance, feature importance, and why ensemble methods excel on tabular datasets. The process was repetitive but satisfying, and it showed me how much tuning matters in practical ML.

Results & Analysis

Final Method Summary

After multiple experiments, the final method consisted of:

- **Preprocessing:**
    - One-hot encoding all categorical features
    - Standardizing numerical columns (for MLP only)
    - Stratified train-test splitting
- **Models Evaluated:**
    - Random Forest (tuned)
    - XGBoost (tuned)
    - SMOTE + XGBoost (oversampled experiment)
    - Multi-Layer Perceptron (tuned neural network)
- **Primary Evaluation Metric:**
    - Macro F1-score
      because the dataset is highly imbalanced and accuracy alone is misleading.

Cross-validation was used throughout to ensure reliable and consistent performance estimates.

Model Performance Summary Table

Model	CV Macro F1	Test Accuracy	Precision (Class 1)	Recall (Class 1)	Notes
Random Forest (tuned)	~0.62	0.867	0.46	0.33	Struggles with minority class
XGBoost (tuned)	0.636	0.882	0.56	0.31	Best overall model
SMOTE + XGBoost	~0.67	0.814	0.34	0.49	Best recall for minority class

Model	CV Macro F1	Test Accuracy	Precision (Class 1)	Recall (Class 1)	Notes
MLP (tuned)	Lower & unstable	~0.84	0.25	0.18	Neural network underperformed

Key finding:
- Tuned XGBoost achieved the best balance of accuracy and macro F1-score.
- SMOTE XGBoost achieved the best recall for high-selling games, but at the cost of overall accuracy.

Detailed Results of the Final Models:

Tuned XGBoost (Final Chosen Model)

- Accuracy: 0.882
- Macro F1-score: 0.67
- Recall (High-Sales): 0.31
- Precision (High-Sales): 0.56

```
XGBoost Test Accuracy: 0.8823889739663093

Classification Report:
              precision    recall  f1-score   support

           0       0.91      0.97      0.93      2853
           1       0.56      0.31      0.40       412

    accuracy                           0.88      3265
   macro avg       0.73      0.64      0.67      3265
weighted avg       0.86      0.88      0.87      3265


Confusion Matrix:
[[2755   98]
 [ 286  126]]
```

Interpretation

- The model correctly identifies most non–high-selling games (class 0).

- It correctly identifies around 31% of high-selling games, which is reasonable given the extreme imbalance.
- Precision for high-selling games is moderate (0.56), meaning that when the model predicts “high-selling,” it is often correct.
- Macro F1-score is the highest among all models, showing the best balance between classes.

This confirms that XGBoost is the strongest model for this dataset, which aligns with real-world expectations — gradient boosting algorithms generally outperform others on structured tabular data.

Random Forest (Tuned)

- Accuracy: 0.867
- Macro F1: ~0.66
- Recall (Class 1): 0.33

```
Random Forest Test Accuracy: 0.8673813169984687

Classification Report:
              precision    recall  f1-score   support

           0       0.91      0.94      0.93      2853
           1       0.46      0.33      0.39       412

    accuracy                           0.87      3265
   macro avg       0.69      0.64      0.66      3265
weighted avg       0.85      0.87      0.86      3265


Confusion Matrix:
 [[2696  157]
 [ 276  136]]
```

Interpretation

Random Forest performed well on the majority class but struggled to identify high-selling games. This model is competitive but not as strong as XGBoost, especially in terms of overall macro-level performance.

SMOTE + XGBoost (Imbalance Handling Experiment)

- Accuracy: 0.814
- Recall (Class 1): 0.49
- Precision (Class 1): 0.34
- Macro F1: ~0.64

```
SMOTE XGBoost Accuracy: 0.8143950995405820

Classification Report:
              precision    recall  f1-score   support

           0       0.92      0.86      0.89      2853
           1       0.34      0.49      0.40       412

    accuracy                           0.81      3265
   macro avg       0.63      0.67      0.64      3265
weighted avg       0.85      0.81      0.83      3265


Confusion Matrix:
[[2459  394]
 [ 212  200]]
```

Interpretation

SMOTE significantly improves the model's ability to detect high-selling games:

- Recall increases from 0.31 → 0.49.
- The model becomes much "fairer" to the minority class.
- Accuracy drops (expected), because the training distribution no longer matches real-world data.

This experiment shows the trade-off between overall accuracy and minority-class usefulness.

Multi-Layer Perceptron (MLP Neural Network)

- Accuracy: ~0.84
- Recall (Class 1): 0.18
- Macro F1: low

The neural network underperformed because:

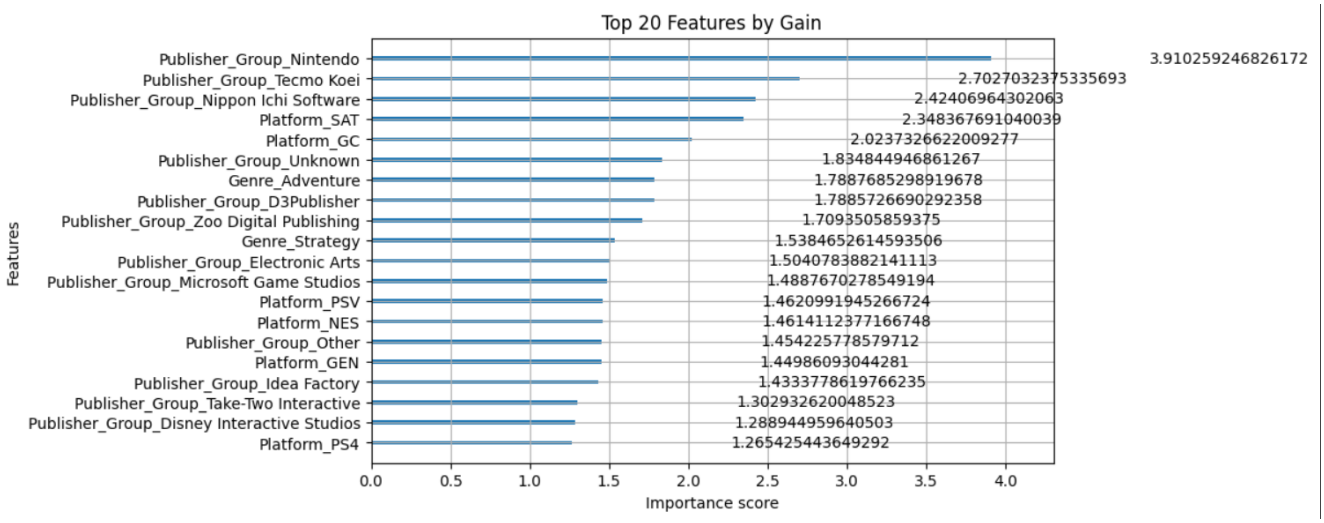
- One-hot encoded data leads to very sparse high-dimensional input.
- Neural nets need huge data or embedded representations to shine.
- Tree-based models naturally handle categorical relationships better.

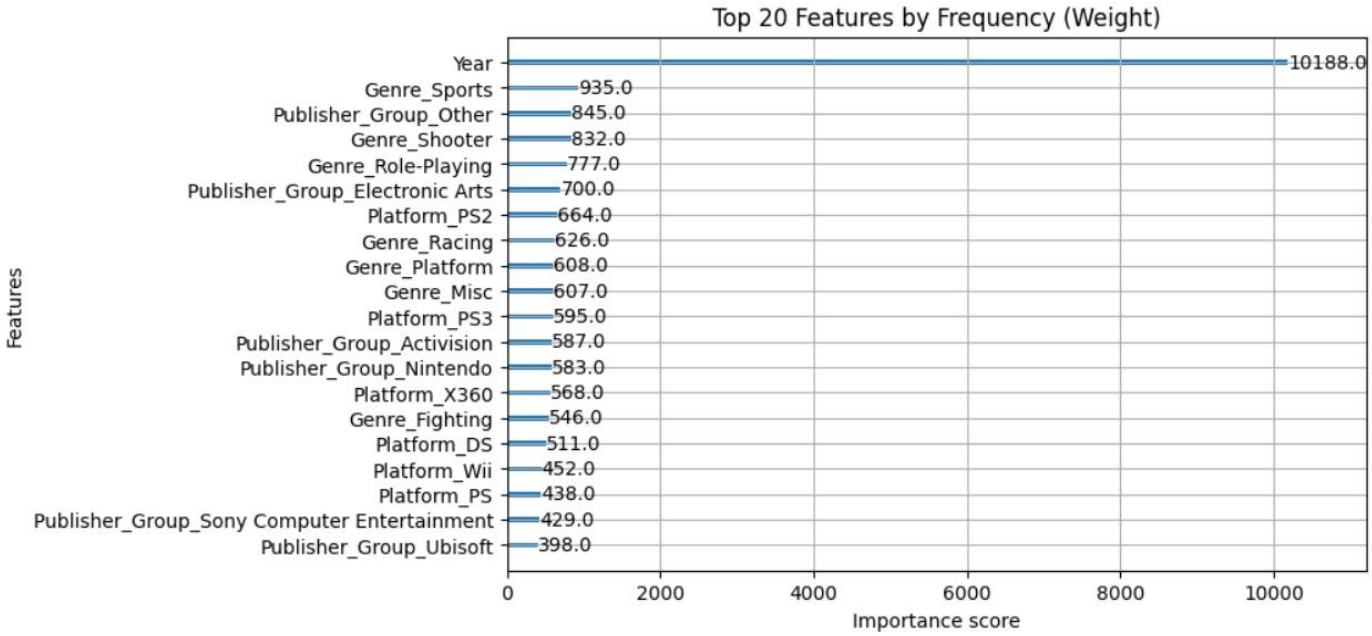
This supports the common ML knowledge that tabular data favors tree-based models over neural networks.

Feature Importance Analysis

Using XGBoost's feature importance:

- Publisher groups were among the most important predictors (Nintendo, Tecmo Koei, Namco Bandai).
- Platform categories were also highly influential (Wii, PS2, DS, etc.).
- Year contributed moderately.
- Genre mattered but was not as dominant as publisher or platform.





Interpretation

These results confirm intuitive industry patterns:

- Major publishers tend to produce consistently high-selling games.
- Platform popularity strongly affects game success.
- Genre helps but does not dominate — action and sports games have wide appeal, but publisher/platform effects are stronger.

What the Results Mean for the Data

From all experiments, I learned:

- Publisher identity is the strongest predictor of high sales.
- Platform matters significantly — games released on widely adopted consoles sell better.
- Genre is secondary, meaning a good publisher/platform combination can overshadow genre preferences.
- Year shows that older eras (e.g., the Wii/DS generation) contained many top-selling titles.

The dataset shows very clear structural patterns:
→ **Sales success is not random; it is heavily tied to industry powerhouses and console popularity.**

Potential Practical Applications

This type of machine learning model could be useful for:

- Game developers estimating the commercial potential of new titles.
- Publishers deciding which platforms to focus on.
- Marketing teams adjusting budgets based on expected sales.
- Investors analyzing market trends.
- Retailers forecasting demand for physical copies (historically relevant).

If combined with modern gaming data (2020–2024), this model could help predict:

- Digital vs physical performance
- Impact of franchise reputation
- Subscription services (Game Pass, PS Plus)
- Live-service and free-to-play trends

Because this dataset is old, the patterns reflect the gaming industry during the PS2/Wii/DS era. However, the methodology and machine learning approach remain relevant today.

References

1. Kaggle. Video Game Sales Dataset. Available from: https://www.kaggle.com/datasets/gregorut/videogamesales
2. Statista. Global Gaming Market Trends 2015–2024. Available from: https://www.statista.com/topics/3070/video-game-industry/
3. MathWorks. Types of Machine Learning Models Explained. Available from: https://www.mathworks.com/discovery/machine-learning-models.html